



D.Y. Patil Academic Education Excellence Federation's

Dr. D. Y. Patil Technical Campus

Polytechnic | Engineering | MCA

(Approved by AICTE, Recognized by DTE, Govt. of Maharashtra, Affiliated to Savitribai Phule Pune University, Pune)

Sr.No.32/1/A/7, Near Talegaon Railway Station, A/P Varale, Talegaon Dabhade, Tal-Maval, Dist-Pune 410 507

Website: www.dypatiltechs.com Email: principal@dypatiltechs.com

AICTE ID: 1- 43358498993 | SPPU PUN CODE: CMMP021930 | ISHE CODE: C -71504 | DTE CODE: 6991

Dr. D. Y. Patil
Founder Dr. D. Y. Patil Group

Dr. Sushant V. Patil
Chairman

Adv. Anuja Sushant Patil
Director

Dr. Suresh Shirbahadurkar
Principal

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

TOPIC: Git and GitHub

Prepared By: Tanvi Mhatungade

No.	Topic	Main Areas Covered
01	Introduction to Version Control	Meaning, importance and purpose
02	Centralized vs. Distributed VCS	Architecture, working and comparison
03	Git Architecture & Three States	Working directory, staging area and repository
04	Core Git Commands & Workflow	Init, clone, add, commit, status and log
05	Git Branching & Merging	Fast-forward, 3-way merge and merge conflicts
06	Git Undoing & History Modification	Checkout, reset, revert, restore and clean
07	Introduction to GitHub	Remote hosting, repositories, forks and stars
08	Remote Collaboration Workflows	Push, pull, fetch, remotes and tracking branches
09	Pull Requests & Code Review	Branch protection, PR reviews and collaboration
10	Advanced Git Features	Rebase, stash, cherry-pick and bisect
11	GitHub Actions & CI/CD	Automation, workflows, runners and deployment
12	Best Practices & Common Errors	Commit hygiene, .gitignore and troubleshooting
13	Conclusion	Key points and importance of Git & GitHub



D.Y. Patil Academic Education Excellence Federation's

Dr. D. Y. Patil Technical Campus

Polytechnic | Engineering | MCA

(Approved by AICTE, Recognized by DTE, Govt. of Maharashtra, Affiliated to Savitribai Phule Pune University, Pune)

Sr.No.32/1/A/7, Near Talegaon Railway Station, A/P Varale, Talegaon Dabhade, Tal-Maval, Dist-Pune 410 507

Website: www.dypatilts.com Email: principal@dypatilts.com

AICTE ID: 1- 43358498993 | SPPU PUN CODE: CMMP021930 | ISHE CODE: C -71504 | DTE CODE: 6991

Dr. D. Y. Patil
Founder Dr. D. Y. Patil Group

Dr. Sushant V. Patil
Chairman

Adv. Anuja Sushant Patil
Director

Dr. Suresh Shirbahadurkar
Principal

INTRODUCTION TO VERSION CONTROL

What is Version Control?

Version control is a system that records changes to a file or set of files over time so that specific versions can be recalled later. In software engineering, Version Control Systems (VCS) track source code modifications, manage project evolution, and maintain an immutable historical record of who made changes, when, and why.

A robust version control system empowers engineering teams to collaborate seamlessly on complex codebases, prevent accidental overwrites, isolate experimental features, and quickly revert regressions to preserve software stability.

Importance of Version Control

- Maintains a complete historical audit trail of all source code modifications.
- Facilitates simultaneous collaboration without risk of overwriting peer contributions.
- Enables instantaneous rollback to previous stable releases upon detecting bugs.
- Supports isolated experimentation through lightweight feature branching.
- Serves as the foundational infrastructure for modern CI/CD and DevOps workflows.

Example

Basic: Storing multiple project copies like project_v1.zip and project_final_v2.zip.

Better: Using a version control system where every change is tracked as an immutable commit with cryptographic hashes, author metadata, and clear messages.

The version-controlled model guarantees codebase integrity, reproducibility, and frictionless team collaboration.



CENTRALIZED VS. DISTRIBUTED VCS

1. Centralized Version Control Systems (CVCS)

A Centralized VCS relies on a single central server that stores the entire versioned history of the codebase. Developers check out individual file snapshots onto their local machines to make changes and commit directly back to the central server.

Examples: Subversion (SVN), Concurrent Versions System (CVS), Perforce.

2. Distributed Version Control Systems (DVCS)

In a Distributed VCS, clients do not just check out the latest snapshot of the files; they fully clone the entire repository, including its complete historical database. Every developer clone is a complete, autonomous backup of the repository.

Examples: Git, Mercurial, Bazaar.

3. Architectural Comparison

Understanding architectural distinctions enables teams to select the appropriate tool for their organizational requirements.

Parameter	Centralized VCS (e.g., SVN)	Distributed VCS (e.g., Git)
Repository Storage	Single central server holds history	Every clone contains full history
Offline Operations	Very limited; requires network connection	Full offline commit, log, and diff capability
Point of Failure	Central server failure halts team work	Any developer clone can restore server
Execution Speed	Slower due to frequent network calls	Ultra-fast; almost all actions are local

Why Distributed Architecture Matters

- Eliminates single-point-of-failure vulnerabilities in mission-critical projects.
- Enables rapid local branching, committing, and diffing without network overhead.
- Provides flexible collaborative workflows tailored to open-source and enterprise teams.



GIT ARCHITECTURE & THREE STATES

Git treats data conceptually as a set of snapshots of a miniature filesystem. Files within a Git-managed project transition across three primary logical areas, each serving a distinct role in the versioning lifecycle.

The Three Primary Areas of Git

Understanding these three sections is crucial for mastering Git's staging and commit mechanisms.

Logical Area	Primary Function & Operational State	Associated Action / State
Working Directory	Local sandbox containing actual files extracted from Git database	Active editing; modified state
Staging Area (Index)	Intermediate binary index formatting what will go into next commit	Staged state via git add
Git Directory (.git)	Permanent object database storing metadata and compressed snapshots	Committed state via git commit

File Lifecycle States in Git

Every file in the working copy resides in either an untracked state or one of three tracked states.

File State	Detailed Description	Transition Mechanism
Untracked	File exists in directory but is not tracked in Git index	Newly created file
Modified	Tracked file altered in workspace but not yet staged	File edited and saved
Staged	Modified file marked in index for inclusion in next commit	Run git add <file>
Committed	Data safely recorded and sealed in local .git database	Run git commit -m

Snapshots, Not Differences

Git does not store files as a set of incremental diffs. Instead, every commit records a complete tree snapshot. To optimize storage, if a file has not changed between commits, Git simply creates a lightweight pointer to the previously stored identical blob.



CORE GIT COMMANDS & WORKFLOW

Standard Git Command Reference

Git provides an intuitive command-line interface that controls the local repository lifecycle, allowing developers to initialize projects, stage discrete modifications, and commit snapshots.

Command	Operational Syntax and Functional Purpose
git init	git init — Initializes a new, empty Git repository and creates the .git metadata directory.
git clone	git clone <url> — Downloads an existing remote repository with its complete commit history.
git status	git status — Inspects the working tree and index, displaying tracked, untracked, and staged files.
git add	git add <file> — Stages modified or newly created files, preparing them for the next commit snapshot.
git commit	git commit -m "message" — Permanently seals staged changes into an immutable commit.
git log	git log --oneline — Displays chronological commit history with abbreviated hashes and messages.

Standard Development Workflow

Day-to-day engineering follows a structured, iterative progression from raw editing to verified commit.

Workflow Phase	Action Executed	Resulting Internal State
1. Modification	Edit code files in the local IDE	Files marked as Modified in working tree
2. Staging	Run git add <target>	Selected changes cached into the staging Index
3. Committing	Run git commit -m "feat: add login"	New commit object created; HEAD updated
4. Verification	Run git status and git log	Working tree confirmed clean; commit verified

Why Atomic Commits Matter

- Keeps each commit focused strictly on a single logical change or bug fix.
- Simplifies code review, debugging, and targeted rollbacks via git revert.
- Prevents tangled pull requests and drastically reduces merge conflict complexity.



GIT BRANCHING & MERGING

Branching Mechanics in Git

In Git, a branch is simply an ultra-lightweight, movable 41-byte pointer to a specific commit. Creating, switching, and deleting branches incurs near-zero overhead, encouraging isolated feature development.

Branch Command	Operational Syntax	Effect on Repository Pointer
git branch <name>	git branch feature-auth	Creates a new branch pointer without switching to it.
git checkout -b	git checkout -b feature-auth	Creates a new branch and immediately switches HEAD to it.
git switch	git switch main	Modern command to switch active branches safely.
git branch -d	git branch -d feature-auth	Safely deletes a branch that has already been merged.

Merge Strategies & Conflict Resolution

Merging integrates independent lines of development into a single branch. Git selects merge strategies based on history topology.

Merge Strategy	Underlying Mechanism	Operational Characteristics
Fast-Forward	Target branch is a direct ancestor with no divergent commits	Simply advances pointer; linear history without merge commit.
3-Way Merge	Combines two divergent branches using their common ancestor	Creates a dedicated merge commit with two parent references.
Merge Conflict	Competing changes modify the exact same lines in both branches	Halts merge; requires developer to manually resolve conflicts.

Example Conflict Resolution

Conflict Markers: Git inserts standard indicators: <<<<<< HEAD (current), =====, and >>>>>> branch (incoming).

Resolution Workflow: The engineer opens the file, resolves the conflicting logic, removes all conflict markers, stages the resolved file with git add, and executes git commit to complete the merge.



GIT UNDOING & HISTORY MODIFICATION

What is History Modification?

Git provides sophisticated mechanisms to unstage files, discard uncommitted edits, revise commit messages, or completely reverse commits. Choosing the correct undo mechanism is essential to prevent unintended code loss.

Comparison of Reset, Revert, and Restore

Command	Scope of Operation	Impact on Commit History
git restore <file>	Discards unstaged modifications in the working tree	No history change; restores file to latest commit state.
git restore --staged	Removes a file from the staging area back to working tree	No history change; preserves working directory edits.
git reset --soft	Moves HEAD back to commit; keeps all changes staged	Rewrites history locally; changes ready to re-commit.
git reset --hard	Resets HEAD, staging index, and working tree to commit	Destructive; permanently discards uncommitted work.
git revert <hash>	Generates a new commit that inverses the target commit	Non-destructive; perfectly safe for shared remote branches.

Steps to Safely Undo Mistakes

- Use `git restore <file>` to discard accidental local changes in the working tree.
- Use `git restore --staged <file>` to unstage files accidentally added via `git add ..`
- Use `git revert <commit>` on public branches to preserve complete audit trails.
- Reserve `git reset --hard` strictly for unpushed, local experimental branches.

Why Safe Undoing is Essential

In collaborative environments, rewriting pushed history with destructive commands like `reset` disrupts teammates' local repositories. Employing non-destructive techniques like `revert` ensures seamless team synchronization.



INTRODUCTION TO GITHUB

What is GitHub?

GitHub is a cloud-based hosting platform built on top of Git. It combines Git's distributed version control engine with a rich web interface, access control governance, issue tracking, project management, and automated CI/CD tools.

Core GitHub Features and Capabilities

GitHub Feature	Technical Description	Primary Engineering Use Case
Remote Repository	Central cloud-hosted Git repository for team sync	Single source of truth for engineering teams.
Forks	Server-side personal clone of another user's repository	Open-source contributions and independent testing.
Issues & Projects	Integrated issue tracking and Kanban board management	Tracking bugs, feature requests, and sprint roadmaps.
Pull Requests	Formal peer-review mechanism for proposed code changes	Code review, CI testing, and branch governance.
GitHub Pages	Static website hosting served directly from a repository	Hosting project documentation, portfolios, and blogs.

Key Collaboration Concepts

- **Remote Synchronization:** Connects isolated developer workstations to a shared cloud repository.
- **Role-Based Access:** Enforces fine-grained permissions (Read, Triage, Write, Maintain, Admin).
- **Organization Workspaces:** Groups related enterprise repositories under unified billing and policies.
- **Open-Source Ecosystem:** Empowers discovery, starring, discussions, and developer sponsorships.

Benefit

GitHub bridges the gap between individual programming and collective software delivery, turning Git repositories into collaborative hubs that streamline peer reviews, automated validation, and continuous deployment.



REMOTE COLLABORATION WORKFLOWS

Remote Management Fundamentals

Collaborating via Git requires managing connections to remote repositories. A 'remote' is simply a named URL alias (traditionally origin) pointing to a shared repository hosted on GitHub or another Git server.

Core Remote Commands

Command	Operational Syntax	Primary Role in Workflow
git remote add	git remote add origin <url>	Links a local repository to a remote repository URL.
git push	git push -u origin <branch>	Uploads local commits to the remote branch and sets upstream.
git fetch	git fetch origin	Downloads remote commits and refs without modifying working tree.
git pull	git pull origin <branch>	Fetches remote changes and immediately merges into active branch.
git remote -v	git remote -v	Lists all configured remote aliases and their fetch/push URLs.

Fetch vs. Pull Distinction

A common point of confusion is the difference between git fetch and git pull. git fetch downloads new data from the remote without touching your local files. In contrast, git pull is shorthand for git fetch followed immediately by git merge.

Guidelines for Clean Remote Collaboration

- Always run git pull --rebase before pushing to prevent unnecessary merge bubbles.
- Never force push (git push --force) to shared branches like main or develop.
- Use the -u flag on initial push to configure upstream tracking for effortless future pulls.



PULL REQUESTS & CODE REVIEW

What is a Pull Request (PR)?

A Pull Request is a collaborative proposal on GitHub to merge code from a feature branch into a target branch. It provides a dedicated forum where engineers discuss implementations, conduct code reviews, and run automated tests.

The Pull Request Lifecycle

Lifecycle Phase	Primary Engineering Activities	Key Quality Objective
1. Creation	Open PR, specify base/compare branches, document changes	Communicate context and rationale to reviewers.
2. Automated CI	GitHub Actions runs test suites, linters, and security checks	Catch syntax errors, broken tests, and security flaws.
3. Peer Review	Teammates inspect diffs, comment on lines, request changes	Enforce architecture standards and optimize logic.
4. Revision	Author pushes commits addressing feedback to the same branch	Iteratively refine code until all criteria are met.
5. Merging	Merge into target branch via Squash, Rebase, or Merge commit	Incorporate feature smoothly into production stream.

Branch Protection Rules

- Require a minimum number of peer review approvals before a PR can be merged.
- Require status checks (unit tests, build checks) to pass successfully before merging.
- Prohibit direct pushes and force pushes to critical branches like main.

Why Code Reviews Matter

Code reviews enhance software quality, eliminate defects before production deployment, foster shared code ownership, and accelerate knowledge transfer across junior and senior engineers.



ADVANCED GIT FEATURES

Power Tools for Version Control

Beyond standard commit and branch operations, Git offers powerful specialized tools for history rewriting, temporary state shelving, isolated commit extraction, and binary debugging.

Command	Operational Syntax	Primary Engineering Application
git rebase	git rebase main	Reapplies commits on top of another base tip; produces linear history.
git stash	git stash push / pop	Temporarily shelves uncommitted modifications to switch tasks cleanly.
git cherry-pick	git cherry-pick <hash>	Applies a specific individual commit from one branch to another.
git bisect	git bisect start / bad / good	Performs binary search across commits to pinpoint bug regressions.
git reflog	git reflog	Records every movement of HEAD; recovers lost commits or branches.

Rebase vs. Merge Comparison

Choosing between rebasing and merging impacts repository history structure and traceability.

Comparison Factor	Git Merge	Git Rebase
History Topology	Preserves true non-linear history with branches	Flattens commits into a single clean, linear line
Commit Identity	Preserves original commit hashes exactly	Recreates commits with brand-new SHA hashes
Best Use Case	Merging finished features into shared branches	Updating local feature branch with latest main

Good Practice

- The Golden Rule of Rebase: Never rebase commits that have already been pushed to a public, shared repository.
- Use git stash to quickly save work-in-progress when an urgent production hotfix demands immediate attention.



D.Y. Patil Academic Education Excellence Federation's

Dr. D. Y. Patil Technical Campus

Polytechnic | Engineering | MCA

(Approved by AICTE, Recognized by DTE, Govt. of Maharashtra, Affiliated to Savitribai Phule Pune University, Pune)

Sr.No.32/1/A/7, Near Talegaon Railway Station, A/P Varale, Talegaon Dabhade, Tal-Maval, Dist-Pune 410 507

Website: www.dypatiltechs.com Email: principal@dypatiltechs.com

AICTE ID: 1- 43358498993 | SPPU PUN CODE: CMMP021930 | ISHE CODE: C -71504 | DTE CODE: 6991

Dr. D. Y. Patil
Founder Dr. D. Y. Patil Group

Dr. Sushant V. Patil
Chairman

Adv. Anuja Sushant Patil
Director

Dr. Suresh Shirbahadurkar
Principal

GITHUB ACTIONS & BEST PRACTICES

GitHub Actions for CI/CD Automation

- Define automated workflows using YAML configuration files stored in .github/workflows/.
- Trigger automated jobs upon repository events such as push, pull_request, or scheduled cron.
- Execute unit tests, integration test suites, and linters across Ubuntu, Windows, and macOS virtual runners.
- Build, package, and deploy container images or binaries automatically to cloud production environments.
- Protect sensitive credentials, tokens, and deployment secrets using encrypted GitHub Repository Secrets.
- Leverage verified actions from GitHub Marketplace to assemble complex CI/CD pipelines rapidly.

Common Git Mistakes and Corrections

Common Error	Underlying Cause	Recommended Correction
Committed secrets / keys	Omitted .gitignore rule	Purge via git-filter-repo and rotate credentials immediately.
Detached HEAD state	Checked out a commit hash directly	Create a branch via git switch -c <new-branch> to save work.
Merge conflict panic	Divergent edits on identical lines	Inspect conflict markers, reconcile code, stage, and commit.
Typo in commit message	Rushed git commit message	Run git commit --amend -m "fixed msg" before pushing.
Committed large binaries	Tracked build artifacts / node_modules	Add to .gitignore, then untrack via git rm --cached.